早稲田大学
WASEDA University

WASEDA University
Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫　鶴鳴
SUN Heming

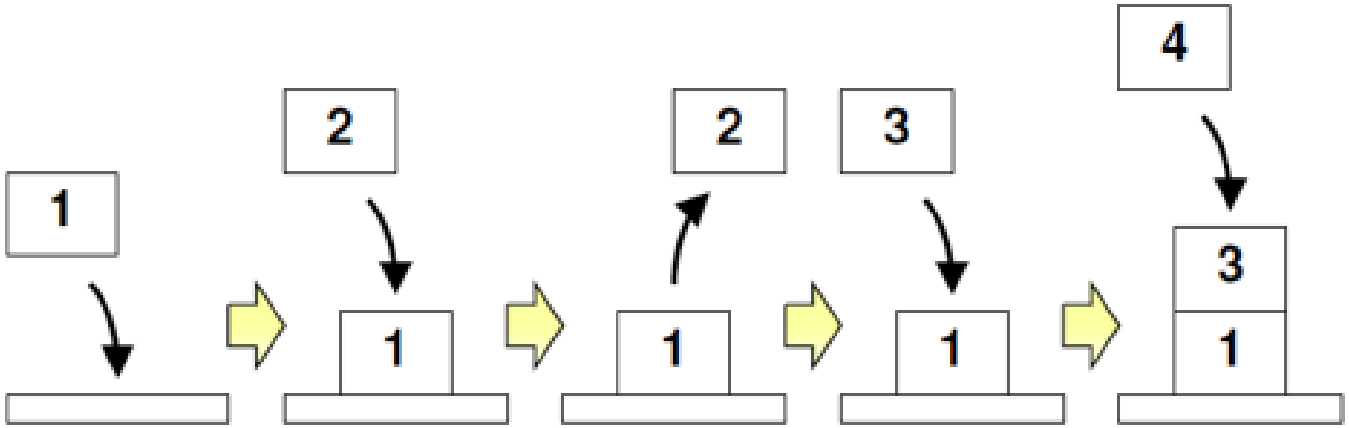
May 22nd

Outline

- Stack
- Queue
- Exercise

Stack

- What is the stack?
- Last In First Out（LIFO）



Stack (cont.)

- Array : The size of array is the number of data in the stack
- Pointer: for stack_top

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define STACK_MAX 10
4 double data[STACK_MAX];
5 int stack_top=0;
```

- Global Variables : declared out of the function and can be used by all functions. data[STACK_MAX], stack_top
- Local Variables: declared within the function and can only be used by this function

Global Variable and Local Variable

- Global Variables
  - Declared outside all functions
  - Can be used by any function
- Local Variables
  - Declared inside a function
  - Can only be used by this function

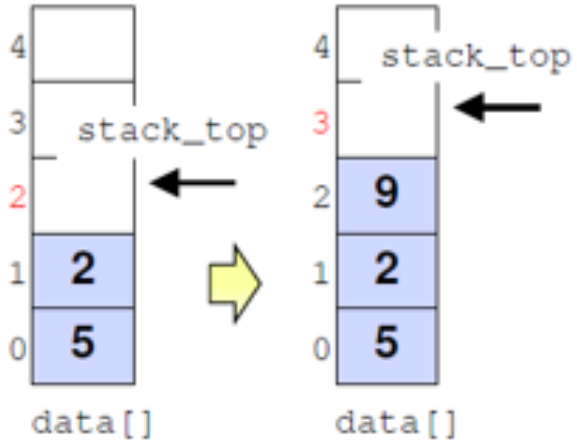
```c
#include <stdio.h>

int a ;

void function(){
  a = 20;
  printf("in func %d¥n", a);
}

int main( ){

  a = 10;
  function();

  printf("in main %d¥n", a);
}
```

```c
#include <stdio.h>

void function(){
   int a = 20;
   printf("in func %d¥n", a);
}

int main( ){

   int a = 10;
   function();

   printf("in main %d¥n", a);
}
```

Push

- Push: put data in the stack
- Put the data at the top of the stack and update the position of (stack_top)

data[stack_top] = value;
stack_top++;




```
6  void stack_push( double value ){
7      data[stack_top] = value;
8      stack_top++;
9  }
```

Pop

- Pop : pop data from the stack
- Decease stack_top by one

  stack_top--;

```
10 double stack_pop( void ){
11     stack_top--;
12     return data[stack_top];
13 }
```

Stack Overflow

- When the stack is full,
- stack overflow

```
5  void stack_push( double value ){
6      if( stack_top == STACK_MAX ){
7          printf( "error: stack overflow\n" );
8          exit(1);
9      }
10     data[stack_top] = value;
11     stack_top++;
12 }
```

Stack Underflow

- When the stack is empty

  stack underflow

```
13 double stack_pop( void ){
14     if( stack_top == 0 ){
15         printf( "error: stack underflow\n" );
16         exit(1);
17     }
18     stack_top--;
19     return data[stack_top];
20 }
```

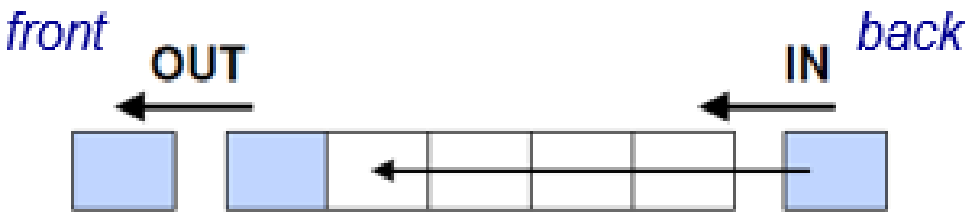
Outline

- Stack
- Queue
- Exercise

Queue

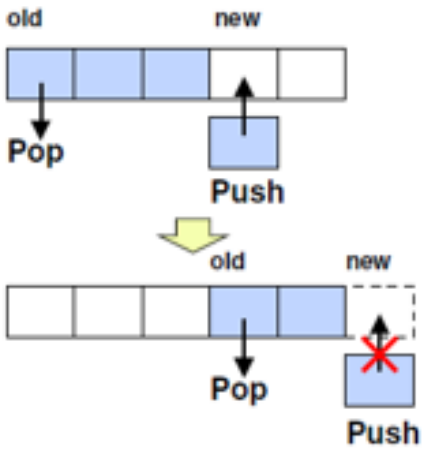
- What is the queue?
- First In First Out(FIFO)

Can only add to back and remove from front

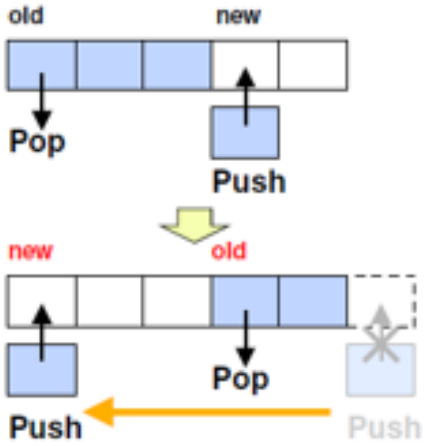


Queue

- Problems of Queueing



- Link Buffer



Queue (cont.)

- Array : The size of array is the number of data in the queue
- Pointer: for head and tail of the queue

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define QUEUE_MAX 10
4 int queue[QUEUE_MAX+1];
5 int queue_head=0, queue_tail=0;
```

- Global Variables
- Size of queue = the number of data +1
- queue_head=0; queue_tail=0;

Enqueue

- Push/Enqueue: add data in the queue
- Put the data at the end of the queue and update the position of (queue_tail)

```
6  void enqueue( int data ){
7      queue[queue_tail] = data;
8      queue_tail = (queue_tail + 1) % (QUEUE_MAX + 1);
9  }
```

Dequeue

- Pop/Dequeue: read/take data from the queue
- take the first data in the queue and update the position of (queue_head)

```
10 int dequeue( void ){
11     int data;
12     data = queue[queue_head];
13     queue_head = (queue_head + 1) % (QUEUE_MAX + 1);
14     return data;
15 }
```

Queue Full

- When the queue is empty,

```
6  void enqueue( int data ){
7      if( (queue_tail+1)%(QUEUE_MAX+1) == queue_head ){
8          printf( "error: no more space in queue\n" );
9          exit(1);
10     }
11     queue[queue_tail] = data;
12     queue_tail = (queue_tail + 1) % (QUEUE_MAX + 1);
13 }
```

Queue Empty

- When the queue is full

```
14 int dequeue( void ){
15     int data;
16     if( queue_head == queue_tail ){
17         printf( "error: empty queue\n" );
18         exit(1);
19     }
20     data = queue[queue_head];
21     queue_head = (queue_head + 1) % (QUEUE_MAX + 1);
22     return data;
23 }
```

Outline

- Stack
- Queue
- Exercise